

PROJECT #2 HUFFMAN CODING

CSC 172 (Data Structures and Algorithms) Fall 2024

University of Rochester

Due Date: Sunday November 24th 2024 @ end of day

Objectives

This project covers a lot of topics, which includes but not limited to, priority queues, maps, and trees. Also you will learn how to read and write binary files. Most importantly, you are going to learn Huffman coding, a lossless compression algorithm and an optimal method for symbol encoding.

Project Description

Huffman encoding is one of the most popular lossless compression algorithms that works fairly well on any text file. In fact, this algorithm can be applied to any file. It can reduce the storage required by a half or even more in some situations. At the end of the project, you will impress yourself implementing this excellent algorithm! This particular project can be used to even encrypt and decrypt files (provided you keep the frequency file hidden).

In this project, you have to write a program that allows the user to compress and decompress files using the standard Huffman algorithm for encoding and decoding.

Overview of the Program Structure

We will provide four Java files to start with:

BinaryIn.java

(source: <https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/BinaryIn.java.html>)

This class provides methods for reading bits (one or many) from a binary input stream. The binary input stream can be produced from a filename.

The methods of interest are:

```
public boolean readBoolean();
public char readChar();
```

BinaryOut.java

(source: <https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/BinaryOut.java.html>)

This class provides methods for writing 1-bit boolean or 8-bit char to an output stream such as a file.

The methods of interest:

```
public void write(boolean x);
public void write(char x);
public void flush(); // Really important. Flushes the binary output
    stream, padding 0s if number of bits written so far is not a
    multiple of 8.
```

Huffman.java

Huffman.java contains Huffman interface which you must implement.

You are not allowed to modify this file.

This interface contains two methods:

```
public void encode(String inputFile, String outputFile, String freqFile);  
public void decode(String inputFile, String outputFile, String freqFile);
```

HuffmanSubmit.java

This class implements Huffman interface (given in Huffman.java). All modifications that you make must be limited to this file. Feel free to add new methods and variables, and import any packages as required. The only requirement is implementing encode and decode methods correctly. We will test your submission on this two methods only.

source code

You can download the zip file from https://www.cs.rochester.edu/u/pawlicki/csc/172/Fall2022/proj2_upload.zip

Extract the tar file. The resulting directory should contain four java files and 2 sample files to test your code. Both BinaryIn.java and BinaryOut.java files contain main methods to play with the functionality of these two classes.

Your Task

HuffmanSubmit.java

This is the only file where you need to do any modification.

Implement encode and decode methods.

Input arguments for encode method:

- *inputFile*: The name of the file to be encoded
- *outputFile*: The name of the encoded file. This is the compressed file (produced after encoding)
- *freqFile*: The name of the frequency file. This file stores the frequency of each character present in the input file. Each row contains a 8 digit representation of the char and its frequency separated by ':'. A sample row may look like: 11001010:40 where '11001010' is the character which has appeared 40 times in the input file.

Input arguments for decode method:

- *inputFile*: The name of the already encoded file.
- *outputFile*: The name of the decoded file you want to produce. This is the file to be written after decoding.
- *freqFile*: The name of the frequency file created while encoding. Use this file to construct the Huffman Tree and perform decoding.

A sample invocation of encode and decode methods

You can test your code with the provided sample files, `alice30.txt` and `ur.jpg`. For your convenience, I have already added the following code snippet to the `HuffmanSubmit.java` file.

```
Huffman huffman = new HuffmanSubmit();
huffman.encode("ur.jpg", "ur.enc", "freq.txt");
huffman.decode("ur.enc", "ur_dec.jpg", "freq.txt");
```

- The encoded file `ur.enc` is input to the decode method
- In encode method, we write '`freq.txt`' and in decode method we read the same file.
- The output file generated in decode method should have a different name than the file you started with (in this example, `ur.jpg` vs. `ur_dec.jpg`). This is just to ensure that decoding, both `ur.jpg` and `ur_dec.jpg` are the same. On linux and mac, you can use '`diff`' command to check if they are the same.
- Important: Encoding and decoding are not required to be done at the same time. For example, I encode a file today. close the program, and tomorrow, I should be able to decode the file. Also, the same program should be able to encode and decode multiple files during the same run, i.e., each encode and decode is independent.

How to submit

Zip (archive) `HuffmanSubmit.java` and a README file together and save it as `Proj2.zip`. Upload the file to Blackboard. Your README file should include any specific instruction which is useful for the project. Make sure your source file are properly commented so that user can browse your code without difficulty. **You will lose points for submitting any additional file or directory.**

Grading Details

- `Proj2.zip` file must contain only two files, `HuffmanSubmit.java` and `README`. (**10 pts** will be deducted for not following the direction)
- Frequency file correctly generated: **30 pts**
- `encode()` method performs correctly: **30 pts**
- `decode()` method performs correctly: **40 pts**
(For getting any credit for decoding, your encoding must work correctly)

You should thoroughly check your code for encoding and decoding before submission.

Acknowledgement

Thanks to Robert Sedgewick and Kevin Wayne for providing `BinaryIn.java` and `BinaryOut.java` files.

Tips and Suggestions

Note: We will update this section based on questions from the students.

- When writing the bit patterns to the encoded file, you do not write the ASCII characters '0' and '1' (That would rather increase the file size), instead the bits are written as true/false (0/1) using the write(boolean) function given in BinaryOut.java.
- Frequency file is a regular text file. So, you really do not need to use BinaryOut for that
- If you prefer tar file over .zip file, you can download it from: https://www.cs.rochester.edu/u/pawlicki/csc/172/Fall2022/proj2_upload.tar
- You are allowed to submit only one file — HuffmanSubmit.java. So, you can create a Node class **inside** HuffmanSubmit class.
Refer: <https://docs.oracle.com/javase/tutorial/java/java00/nested.html>